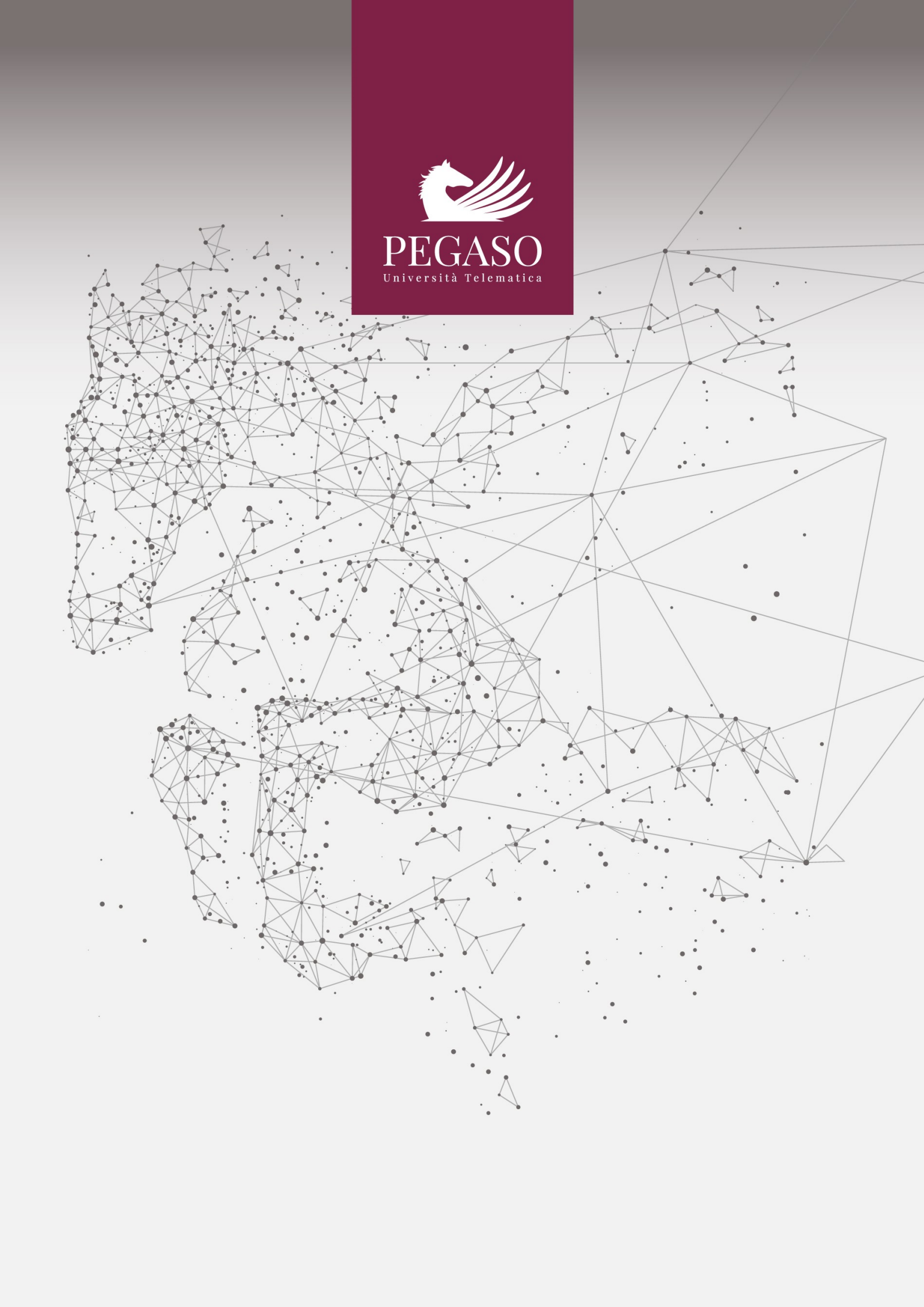

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3
2. FILTRARE E ORDINARE I DATI 4
3. MANIPOLARE ED ESPLORARE DATAFRAME 7
4. CALCOLI E STATISTICHE SU DATAFRAME 12
5. RAGGRUPPAMENTI 17
BIBLIOGRAFIA 22

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si presentano le operazioni fondamentali eseguibili con Pandas. In particolare, si illustrano le operazioni e le funzioni per filtrare e ordinare i dati inclusi in un DataFrame. Successivamente, si presentano le operazioni per manipolare ed esplorare DataFrame con una particolare attenzione all'indicizzazione e alla cancellazione di righe e colonne. Infine, si presentano le operazioni e le funzioni per i calcoli, le statistiche e i raggruppamenti sui dati contenuti nel DataFrame.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Filtrare e ordinare i dati

Con i DataFrame possiamo applicare delle **selezioni** sui dati operando in un modo che ricorda molto quello di NumPy. Risulta essere di fondamentale importanza, quindi, la progettazione della selezione da applicare formulandola all'interno delle parentesi quadre. Si parte dal seguente DataFrame utilizzato già nella precedente lezione.

	nome	cognome	altezza	peso	nascita	sesso
0	Giulio	Rossi	1.84	89	1994-08-12	M
1	Alessia	Verdi	1.62	53	1993-03-07	F
2	Enea	Bianchi	1.78	73	1996-05-06	M
3	Paola	Neri	1.74	61	1999-09-30	F
4	Ilenia	gialli	1.70	70	1998-12-27	F
5	Valerio	Verdi	1.88	88	1996-05-06	M

Per selezionare solo le persone di sesso maschile, è possibile impostare la seguente query di ricerca:

df['sesso'] =='M'

che equivale a dire "seleziona tutte le righe del DataFrame *df* in cui il campo sesso è uguale al valore M" e che viene passata all'interno delle parentesi quadre.

```
print(df[df['sesso']=='M'])
```

	nome	cognome	altezza	peso	nascita	sesso
0	Giulio	Rossi	1.84	89	1994-08-12	M
2	Enea	Bianchi	1.78	73	1996-05-06	M
5	Valerio	Verdi	1.88	88	1996-05-06	M

Per selezionare solo le persone con peso superiore a 70 kg (indipendentemente dalle altre caratteristiche) è possibile impostare la seguente query di ricerca:

df['peso'] >70

```
print(df[df['peso']>70])
```

	nome	cognome	altezza	peso	nascita	sesso
0	Giulio	Rossi	1.84	89	1994-08-12	M
2	Enea	Bianchi	1.78	73	1996-05-06	M
5	Valerio	Verdi	1.88	88	1996-05-06	M

Ovviamente è possibile esprimere combinazioni di elementi attraverso la logica booleana.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nel prossimo esempio si vogliono estrarre tutte le donne alte almeno un metro e settanta centimetri.

```
df2 = df[(df['sesso']=='F') & (df['altezza']>1.7)]
print(df2)
```

```
    nome cognome  altezza  peso    nascita sesso
3  Paola    Neri     1.74    61 1999-09-30     F
```

Un ulteriore esempio potrebbe essere quello di cercare tutti i maschi con cognome Verdi o Rossi. In questo caso, la condizione combinata risulta essere più complessa della precedente e vede un OR logico sulle condizioni dei cognomi (Verdi o Rossi) e tale condizione composta in AND logico con la condizione sul sesso maschile.

```
df2 = df[((df['cognome']=='Verdi') | (df['cognome']=='Rossi')) \
         &(df['sesso']=='M')]
print(df2)
```

```
      nome cognome  altezza  peso    nascita sesso
0   Giulio   Rossi     1.84    89 1994-08-12     M
5  Valerio   Verdi     1.88    88 1996-05-06     M
```

I dati presenti nei DataFrame possono essere ordinati utilizzando principalmente i metodi:

- **sort_index**() che restituisce una versione del DataFrame ordinata **in base agli indici** in senso crescente (o decrescente se argomento *ascending = False*);
- **sort_values**() che restituisce una versione del DataFrame ordinato **in base ai valori di una o più colonne** in senso crescente (o decrescente se argomento *ascending = False*).

Nell'esempio seguente si ordinano in ordine decrescente le righe del DataFrame in base all'indice (etichetta).

```
df_index = df.sort_index(ascending= False)
print(df_index)
```

```
      nome  cognome  altezza  peso    nascita sesso
5  Valerio    Verdi     1.88    88 1996-05-06     M
4   Ilenia   gialli     1.70    70 1998-12-27     F
3    Paola     Neri     1.74    61 1999-09-30     F
2     Enea  Bianchi     1.78    73 1996-05-06     M
1  Alessia    Verdi     1.62    53 1993-03-07     F
0   Giulio    Rossi     1.84    89 1994-08-12     M
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nell'esempio seguente si ordinano in ordine alfabetico le righe del DataFrame in base ai valori della colonna nome.

```
df_index = df.sort_values(by='nome')
print(df_index)
```

```
      nome  cognome  altezza  peso    nascita sesso
1  Alessia    Verdi     1.62    53 1993-03-07     F
2     Enea  Bianchi     1.78    73 1996-05-06     M
0   Giulio    Rossi     1.84    89 1994-08-12     M
4   Ilenia   gialli     1.70    70 1998-12-27     F
3    Paola     Neri     1.74    61 1999-09-30     F
5  Valerio    Verdi     1.88    88 1996-05-06     M
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Manipolare ed esplorare DataFrame

Sul DataFrame è possibile svolgere operazioni di manipolazione mirate ad **aggiungere, rimuovere e modificare elementi**. Per mostrare tali funzionalità partiamo da un nuovo DataFrame.

```
import pandas as pd

df = pd.DataFrame(data = [['Luca',28,'Milano'],\
                          ['Alessia',22,'Venezia'],\
                          ['Silvia',24,'Messina'],\
                          ['Filippo',31,'Bari']],\
                  columns=['nome','eta','citta'])
print(df)
```

```
      nome  eta    citta
0     Luca   28   Milano
1  Alessia   22  Venezia
2   Silvia   24  Messina
3  Filippo   31     Bari
```

Procediamo con la modifica della riga di nome (*etichetta*) *1* come mostrato nel codice seguente. A seguito della modifica, i dati di Alessia non saranno più presenti perché sovrascritti dalle informazioni di Paola.

```
df.loc[1]=['Paola',25,'Genova']
print(df)
```

```
      nome  eta    citta
0     Luca   28   Milano
1    Paola   25   Genova
2   Silvia   24  Messina
3  Filippo   31     Bari
```

La stessa operazione è possibile utilizzando la posizione della riga come mostrato di seguito in cui si agisce sull'ultima posizione (*iloc[-1]*).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
df.iloc[-1]=['Paola',25,'Genova']
print(df)
```

```
      nome  eta    citta
0     Luca   28   Milano
1  Alessia   22  Venezia
2   Silvia   24  Messina
3    Paola   25   Genova
```

Quando si utilizza, però, **loc con un indice non presente** nel DataFrame, si ottiene la creazione di una nuova riga. Nel caso seguente, si utilizza l'etichetta 50 non presente tra le etichette. Ne risulta che al DataFrame originale viene aggiunta una riga.

```
df.loc[50]=['Paola',25,'Genova']
print(df)
```

```
       nome  eta    citta
0      Luca   28   Milano
1   Alessia   22  Venezia
2    Silvia   24  Messina
3   Filippo   31     Bari
50   Paola   25   Genova
```

Dall'esempio precedente risulta chiaro che a volte il problema è quello di sapere l'**indice numerico di valore massimo** utilizzato nel DataFrame oppure l'**ultimo indice numerico inserito** (che a volte, come in questo caso, possono coincidere). Per conoscere tali valori è possibile utilizzare index e rispettivamente il metodo *max*() e l'indice -1 come mostrato nel codice seguente.

```
imax = df.index.max()
ilast = df.index[-1]
print("Indice numerico di valore massimo:",imax)
print("Ultimo indice numerico inserito:",ilast)
```

```
Indice numerico di valore massimo: 50
Ultimo indice numerico inserito: 50
```

La cancellazione di elementi in un DataFrame viene offerta da Pandas attraverso il metodo **drop**() che restituisce un nuovo DataFrame con gli elementi cancellati. Il metodo *drop*() richiede il **riferimento da cancellare** e l'argomento **axis** che specifica se si tratta di:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- una riga (*axis = 0*);
- una colonna (*axis = 1*).

Nell'ultimo DataFrame utilizzato, procediamo con la cancellazione della riga (*axis=0*) con etichetta 50 come indicato nel codice seguente.

```
df = df.drop(50, axis =0)
print(df)
```

```
      nome  eta    citta
0     Luca   28   Milano
1  Alessia   22  Venezia
2   Silvia   24  Messina
3  Filippo   31     Bari
```

Successivamente, procediamo con la cancellazione della colonna (*axis = 1*) citta come indicato nel codice seguente.

```
df = df.drop('citta', axis =1)
print(df)
```

```
      nome  eta
0     Luca   28
1  Alessia   22
2   Silvia   24
3  Filippo   31
```

Per la **cancellazione di una riga per posizione**, invece, è necessario usare **index** per ottenere la sua etichetta. Per esempio, la cancellazione dell'ultima riga (*posizione = -1*) corrisponderà al seguente codice:

```
df = df.drop(df.index[-1])
print(df)
```

```
      nome  eta    citta
0     Luca   28   Milano
1  Alessia   22  Venezia
2   Silvia   24  Messina
3  Filippo   31     Bari
```

Pandas offre la possibilità di concatenare due DataFrame attraverso il metodo **concat**().

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Quando si usa il metodo *concat*() però bisogna considerare il reset degli indici dei due DataFrame che si concatenano. Se non fossero resettati attraverso il metodo *reset_index(drop=True)*, infatti, nel nuovo DataFrame ci sarebbero indici doppi. Il codice che permette di concatenare due DataFrame (df1 e df2) è riportato di seguito.

```
import pandas as pd

df1 = pd.DataFrame(data = [['Luca',28,'Milano'],\
                           ['Alessia',22,'Venezia'],\
                           ['Silvia',24,'Messina'],\
                           ['Filippo',31,'Bari']],\
                   columns=['nome','eta','citta'])

df2 = pd.DataFrame(data = [['Giorgio',42,'Roma'],\
                           ['Alessandra',33,'Palermo'],\
                           ['Anna',22,'Napoli'],\
                           ['Vittorio',26,'Trieste']],\
                   columns=['nome','eta','citta'])

df = pd.concat([df1,df2]).reset_index(drop=True)

print(df)
```

```
         nome  eta    citta
0        Luca   28   Milano
1     Alessia   22  Venezia
2      Silvia   24  Messina
3     Filippo   31     Bari
4     Giorgio   42     Roma
5  Alessandra   33  Palermo
6        Anna   22   Napoli
7    Vittorio   26  Trieste
```

Per quanto riguarda l'**esplorazione** dei DataFrame, oltre all'utilizzo dei cicli accedendo alle posizioni con **loc** e **iloc**, Pandas mette a disposizione due metodi molto interessanti:

- **iteritems**(), permette di ottenere un generatore che restituirà ad ogni invocazione un indice di colonna e la Series che contiene i valori della colonna (scorrimento del frame per colonne);
- **iterrows**(), permette di ottenere un generatore che ad ogni iterazione fornirà un indice di riga e la Series contenente i suoi valori (scorrimento del frame per righe).

A titolo esemplificativo, viene riportato un esempio di applicazione per ciascuno dei due metodi appena analizzati. In particolare, con *iteritems*() si vogliono stampare tutti e solo i nomi presenti nel DataFrame, mentre con *iterrows*() si vogliono visualizzare indice, nome e città di ciascuna riga.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import pandas as pd

df = pd.DataFrame(data = [['Luca',28,'Milano'],\
                          ['Alessia',22,'Venezia'],\
                          ['Silvia',24,'Messina'],\
                          ['Filippo',31,'Bari']],\
                  columns=['nome','eta','citta'])

for i, v in df.iteritems():
    if i=='nome':
        print(v)

print()

for i, v in df.iterrows():
    print("All'indice",i,"il campo nome contiene",v.nome,\
          "e il campo città contiene",v.citta)
```

```
0       Luca
1    Alessia
2     Silvia
3    Filippo
Name: nome, dtype: object

All'indice 0 il campo nome contiene Luca e il campo città contiene Milano
All'indice 1 il campo nome contiene Alessia e il campo città contiene Venezia
All'indice 2 il campo nome contiene Silvia e il campo città contiene Messina
All'indice 3 il campo nome contiene Filippo e il campo città contiene Bari
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Calcoli e statistiche su DataFrame

Pandas mette a disposizione strumenti per estrarre in maniera unica e veloce le **principali misure statistiche** di un DataFrame attraverso il metodo **describe**() che estrarrà statistiche per i campi numerici. Per mostrare le potenzialità di tale metodo utilizzeremo il seguente DataFrame:

	nome	cognome	altezza	peso	nascita	sesso
0	Giulio	Rossi	1.84	89	1994-08-12	M
1	Alessia	Verdi	1.62	53	1993-03-07	F
2	Enea	Bianchi	1.78	73	1996-05-06	M
3	Paola	Neri	1.74	61	1999-09-30	F
4	Ilenia	gialli	1.70	70	1998-12-27	F
5	Valerio	Verdi	1.88	88	1996-05-06	M

Come è possibile osservare di seguito, il risultato restituito da describe() fornirà le seguenti statistiche per altezza e peso:

- conteggio di valori;
- media;
- deviazione standard;
- minimo;
- 25° percentile;
- 50° percentile;
- 75° percentile;
- massimo.

```
print(df.describe())
```

	altezza	peso
count	6.000000	6.000000
mean	1.760000	72.333333
std	0.094657	14.361987
min	1.620000	53.000000
25%	1.710000	63.250000
50%	1.760000	71.500000
75%	1.825000	84.250000
max	1.880000	89.000000

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il risultato restituito da *describe*() è a sua volta un DataFrame che è interrogabile con i metodi visti come riportato nei tre codici seguenti. Nel primo verifichiamo il tipo di *stat* (risultato di describe) e visualizziamo la Series relativa alla sola altezza.

```
stat = df.describe()
print("Tipo statistiche",type(stat))
print()
print(stat['altezza'])
```

```
Tipo statistiche <class 'pandas.core.frame.DataFrame'>

count    6.000000
mean     1.760000
std      0.094657
min      1.620000
25%      1.710000
50%      1.760000
75%      1.825000
max      1.880000
Name: altezza, dtype: float64
```

Nel secondo, estraiamo da *stat* il valore massimo di altezza e peso.

```
stat = df.describe()
print(stat.loc['max'])
```

```
altezza     1.88
peso       89.00
Name: max, dtype: float64
```

Nel terzo, estraiamo da *stat* il valore massimo solo relativo al peso.

```
stat = df.describe()
print(stat.loc['max']['peso'])
```

```
89.0
```

Ad ogni modo, oltre a *describe*() esistono funzioni singole mirate ad un'unica misurazione, tra cui ci sono:

- *min*() per il minimo;
- *max*() per il massimo;

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- *sum*() per la somma complessiva;
- *cumsum*() per la somma cumulativa;
- *mean*() per la media;
- *median*() per la mediana.

Per mostrare l'applicazione di tali funzioni, creiamo un Dataframe riguardante le entrate e le uscite finanziarie mensili per un anno come mostrato nel codice seguente.

```
import pandas as pd

bilancio = pd.DataFrame(data=[[1500, 450],
                              [1200, 670],
                               [1550, 490],
                               [890, 1520],
                               [710, 500],
                               [1200, 780],
                               [1150, 670],
                               [1480, 540],
                               [700, 780],
                               [950, 800],
                               [1320, 1900],
                                [1950, 1200]],
                             columns=['Entrate','Uscite'],
                              index=['Gennaio',
                                     'Febbraio',
                                     'Marzo',
                                     'Aprile',
                                     'Maggio',
                                     'Giugno',
                                     'Luglio',
                                     'Agosto',
                                     'Settembre',
                                     'Ottobre',
                                      'Novembre',
                                     'Dicembre'])
print(bilancio)
```

```
           Entrate  Uscite
Gennaio       1500     450
Febbraio      1200     670
Marzo         1550     490
Aprile         890    1520
Maggio         710     500
Giugno        1200     780
Luglio        1150     670
Agosto        1480     540
Settembre      700     780
Ottobre        950     800
Novembre      1320    1900
Dicembre      1950    1200
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Procediamo con l'analisi del DataFrame e delle Series con le funzioni viste precedentemente. Per prima cosa analizziamo la somma totale di entrate e uscite come mostrato di seguito. Si ottiene così una Series con gli indici Entrate e Uscite che mostra che le prime ammontano a 14.600, le seconde a 10.300. In totale c'è stato un guadagno annuale di 4.300.

```
print(bilancio.sum())
```

```
Entrate    14600
Uscite     10300
dtype: int64
```

Lo stesso dato di guadagno annuale finale, è calcolabile attraverso la somma cumulativa (*cumsum*()) delle differenze tra entrate e uscite. Il mese di dicembre permette, infatti, di ottenere il risultato cumulativo dell'anno con 4.300.

```
c = (bilancio.Entrate-bilancio.Uscite).cumsum()
print(c)
```

```
Gennaio      1050
Febbraio     1580
Marzo        2640
Aprile       2010
Maggio       2220
Giugno       2640
Luglio       3120
Agosto       4060
Settembre    3980
Ottobre      4130
Novembre     3550
Dicembre     4300
dtype: int64
```

Lo stesso risultato è ottenibile anche in modo diretto analizzando solo la somma cumulativa nel mese di Dicembre.

```
c = (bilancio.Entrate-bilancio.Uscite).cumsum().Dicembre
print(c)
```

```
4300
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Le altre informazioni che si possono estrarre dal DataFrame sono la media, il minimo e massimo di entrate e di uscite come si può osservare nel codice seguente. I commenti indicano come ottenere media, minimo e massimo per una sola delle serie (nell'esempio le entrate).

```
print("Valori minimi")
print(bilancio.min())  #oppure print(bilancio.Entrate.min())\
                       #per le sole entrate
print()
print("Valori massimi")
print(bilancio.max())  #oppure print(bilancio.Entrate.max())\
                       #per le sole entrate
print()
print("Valori medi")
print(bilancio.mean())  #oppure print(bilancio.Entrate.mean())\
                        #per le sole entrate
print()
```

```
Valori minimi
Entrate    700
Uscite     450
dtype: int64

Valori massimi
Entrate    1950
Uscite     1900
dtype: int64

Valori medi
Entrate    1216.666667
Uscite      858.333333
dtype: float64
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Raggruppamenti

Le **operazioni di raggruppamento** sono una delle funzioni chiave dell'elaborazione dei dati.

Corrispondono al **GROUP BY** di **SQL** e permettono di suddividere in gruppi diversi i record in base ai valori assunti da uno o più campi. Partiamo dal DataFrame già usato in precedenza e riportato di seguito.

	nome	cognome	altezza	peso	nascita	sesso
0	Giulio	Rossi	1.84	89	1994-08-12	M
1	Alessia	Verdi	1.62	53	1993-03-07	F
2	Enea	Bianchi	1.78	73	1996-05-06	M
3	Paola	Neri	1.74	61	1999-09-30	F
4	Ilenia	gialli	1.70	70	1998-12-27	F
5	Valerio	Verdi	1.88	88	1996-05-06	M

Il campo sesso del DataFrame basato sul file persone.csv ammette solo due valori M (3 righe) e F (3 righe). Fare un raggruppamento su questo campo significa suddividere idealmente le righe in due categorie. La funzione **groupby**() esegue il raggruppamento il cui risultato può essere osservato tramite la proprietà **groups**. Come si osserva dal risultato, che è un **dizionario Python**, gli elementi di indice 1, 3 e 4 sono finiti nel gruppo femminile, mentre gli elementi di indice 0, 2 e 5 sono finiti nel gruppo maschile.

```
g = df.groupby('sesso').groups
print(g)
```

```
{'F': [1, 3, 4], 'M': [0, 2, 5]}
```

Per accedere alla lista degli indici è possibile accedere a groups con l'etichetta desiderata (es. 'F').

```
gf = df.groupby('sesso').groups['F']
print(gf)
```

```
Int64Index([1, 3, 4], dtype='int64')
```

Per accedere ai singoli indici si procede come visto per le liste in Python. Nel codice seguente vengono mostrate *due modalità*. La prima prevede di iterare su ciascun elemento della lista *gf* e visualizzare a schermo tali indici. Nella seconda, *gf* è già un elemento (il primo corrispondente all'indice 0) e, quindi, viene visualizzato a schermo direttamente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
gf = df.groupby('sesso').groups['F']
for i in gf:
    print("Indice: ",i)
print()
#oppure
gf = df.groupby('sesso').groups['F'][0]
print("Indice: ",gf)
```

```
Indice:  1
Indice:  3
Indice:  4

Indice:  1
```

Ottenuto l'indice (etichetta) della riga è possibile accedere al DataFrame con loc come visto e riportato nel codice seguente.

```
gf = df.groupby('sesso').groups['F'][0]
print(df.loc[gf])
```

```
nome          Alessia
cognome         Verdi
altezza          1.62
peso               53
nascita    1993-03-07
sesso               F
Name: 1, dtype: object
```

Oltre all'esplorazione manuale si può procedere all'iterazione tra gruppi con i meccanismi delle strutture iterative che permettono di accedere alle chiavi e ai corrispondenti valori nel dizionario restituito dalla funzione *groupby*(). Si noti che i valori corrispondenti alle chiavi, identificati nel codice seguente come *elementi_gruppo*, sono a loro volta dei DataFrame gestibili con i metodi analizzati finora.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
for chiave, elementi_gruppo in df.groupby('sesso'):
    print(chiave)
    print(elementi_gruppo)
    print()
```

```
F
      nome cognome  altezza  peso     nascita sesso
1  Alessia   Verdi     1.62    53  1993-03-07     F
3    Paola    Neri     1.74    61  1999-09-30     F
4   Ilenia  gialli     1.70    70  1998-12-27     F

M
      nome  cognome  altezza  peso     nascita sesso
0   Giulio    Rossi     1.84    89  1994-08-12     M
2     Enea  Bianchi     1.78    73  1996-05-06     M
5  Valerio    Verdi     1.88    88  1996-05-06     M
```

Per accedere ad uno solo dei due gruppi, Pandas mette a disposizione la funzione *get_group*() (nell'esempio seguente si estrae solo il gruppo femminile).

```
r = df.groupby('sesso').get_group('F')
print(r)
```

```
      nome cognome  altezza  peso     nascita sesso
1  Alessia   Verdi     1.62    53  1993-03-07     F
3    Paola    Neri     1.74    61  1999-09-30     F
4   Ilenia  gialli     1.70    70  1998-12-27     F
```

Il motivo principale per cui si raggruppano i dati è quello di eseguire *aggregazioni* sui singoli gruppi. Ad esempio, nella suddivisione in gruppo maschile e femminile, potrebbe essere importante calcolare ed estrarre l'altezza media per ogni gruppo o conteggiarne gli elementi.

Le aggregazioni possono essere eseguite in due modi:

1. in caso di **funzionalità standard** come medie, somme, statistiche, è possibile agire con funzioni preimpostate direttamente **dopo il groupby**();
2. in caso contrario si utilizza la funzione **agg**() che permette di impostare funzioni a piacere per l'aggregazione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Procediamo con esempi di operazioni standard:

- conteggio degli appartenenti ai due gruppi

```
r = df.groupby('sesso').size()
print(r)
```

```
sesso
F    3
M    3
dtype: int64
```

- calcolo dell'altezza media dei due gruppi

```
r = df.groupby('sesso')['altezza'].mean()
print(r)
```

```
sesso
F    1.686667
M    1.833333
Name: altezza, dtype: float64
```

- statistiche complessive degli appartenenti ai due gruppi

```
r = df.groupby('sesso')['altezza'].describe()
print(r)
```

```
       count      mean       std   min   25%   50%   75%   max
sesso
F        3.0  1.686667  0.061101  1.62  1.66  1.70  1.72  1.74
M        3.0  1.833333  0.050332  1.78  1.81  1.84  1.86  1.88
```

- estrazione di una singola misurazione del caso precedente (es. max)

```
r = df.groupby('sesso')['altezza'].describe()['max']
print(r)
```

```
sesso
F    1.74
M    1.88
Name: max, dtype: float64
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Procediamo, ora, con esempi di **operazioni non standard** per cui si ricorre ad **agg**() che accetta anche una lista di funzioni che possono essere prese da NumPy (nell'esempio successivo funzioni minimo, massimo, media e deviazione standard di NumPy).

```
r = df.groupby('sesso')['peso'].agg([np.min,np.max,np.mean,np.std])
print(r)
```

```
        amin  amax       mean       std
sesso
F         53    70  61.333333  8.504901
M         73    89  83.333333  8.962886
```

Ovviamente, per le operazioni non standard per cui si ricorre ad agg() è possibile utilizzare anche funzioni personalizzate da passare come **funzioni lambda**. Nel caso seguente, è stata creata una funzione lambda per calcolare il range che separa il massimo dal minimo in cui i valori offerti da x sono quelli presenti nel campo 'peso'.

```
r = df.groupby('sesso')['peso']\
.agg([np.min,np.max,lambda x: np.max(x)-np.min(x)])
print(r)
```

```
        amin  amax  <lambda_0>
sesso
F         53    70          17
M         73    89          16
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Zinoviev D. (2017). Data science con Python. Apogeo.
- Maggi G. (2020). Data science con Python. Edizioni LSWR.
- Marin I., Shukla A., Sarang VK. (2019). L'analisi dei Big Data con Python. Tecniche nuove.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).